Data di consegna: **Martedì 21 Aprile 2026**

INDICE

1. Consegna

2. Requisiti obbligatori

3. Elementi facoltativi per migliorare il lavoro

4. Materiale da consegnare nell'attività Teams

5. Esposizione Orale

6. Griglia di valutazione

7. Suggerimenti operativi

Il progetto consiste nella realizzazione di una **applicazione Java**. L'applicazione deve simulare un piccolo **gestionale**, **videogioco**. Il tema è libero, purché il progetto sia coerente, ben strutturato e permetta di usare realmente gli strumenti del linguaggio Java studiati in classe.

Alcuni esempi possibili:

- gestione biblioteca;
- gestione videoteca;
- agenda scolastica;
- rubrica avanzata;
- gestione ordini e magazzino;
- gestione studenti e corsi;
- gestione prenotazioni;
- catalogo videogiochi, film, libri o musica;
- mini gestionale di noleggio;
- simulatore di e-commerce da terminale;
- Videogioco (esempio: 2048, pac-man, snake, scacchi, dama)
- Altro.

Il progetto può essere svolto:

- individualmente;
- in coppia.

Obiettivo

Progettare e realizzare una applicazione Java a riga di comando che non si limiti a stampare dati, ma che gestisca oggetti, collezioni, menu e operazioni reali su dati strutturati, mostrando una buona padronanza della programmazione orientata agli oggetti.

1. Consegna

Ogni studente o coppia deve ideare e sviluppare una **applicazione in Java**.

L'applicazione deve:

- presentare un **menu testuale** chiaro;
- permettere all'utente di eseguire operazioni concrete;
- utilizzare **classi e oggetti** per rappresentare i dati;
- gestire i dati tramite una o più **collezioni Java**;
- prevedere una struttura del codice ordinata, leggibile e modulare.

Non è sufficiente un programma formato da poche stampe o da esercizi separati: il lavoro deve essere un **progetto unico, coerente e funzionante**, con una logica ben riconoscibile.

2. Requisiti obbligatori

Il progetto deve contenere tutti gli elementi seguenti.

1. Struttura a classi e package

Il programma deve essere organizzato in più classi e package.

2. Programmazione orientata agli oggetti

Devono essere usati in modo corretto:

- attributi;
- modificatori di accesso;
- costruttori;
- metodi get e set quando necessari;
- incapsulamento;
- metodo toString();
- override e overload.

3. Ereditarietà e interfacce

Il progetto deve prevedere **una gerarchia di classi (classi concrete, astratte)** e se necessario **interfacce**.

4. Polimorfismo

Nel progetto devono comparire situazioni in cui oggetti di sottoclassi diverse vengono gestiti tramite un riferimento della superclasse.

5. Uso di collezioni

Il programma deve usare almeno **una struttura dati dinamica**, a scelta tra:

- ArrayList
- HashMap
- LinkedHashMap
- TreeMap
- Stack/Queue/PriorityQueue

Le collezioni devono essere utilizzate per:

- inserire dati;
- cercare dati;
- modificare dati;
- eliminare dati;
- stampare insiemi di dati.

6. Menu interattivo

Il programma deve contenere un **menu testuale**, con più opzioni.

Esempio:

1. inserimento di nuovi dati;
2. visualizzazione dei dati;
3. ricerca di un elemento;
4. modifica oppure eliminazione di un elemento;
5. uscita dal programma.

7. Input utente

L'applicazione deve leggere dati da tastiera, ad esempio con Scanner, e deve guidare l'utente in modo comprensibile.

8. Gestione delle eccezioni

Devono essere gestiti in modo esplicito almeno alcuni errori, ad esempio:

- input numerico errato;
- ricerca di un elemento inesistente;
- file non trovato;
- dati non validi.

Devono essere usati almeno:

- try/catch/throw
- messaggi di errore comprensibili per l'utente

9. Operazioni sui dati

Il programma deve includere almeno alcune operazioni significative sui dati, come:

- conteggi;
- filtri;
- ordinamenti;
- ricerche;
- statistiche semplici;
- controllo duplicati;
- classificazioni per categoria o tipo.

L'obiettivo è dimostrare che lo studente sa usare le strutture dati in modo ragionato.

3. Elementi facoltativi per migliorare il lavoro

I seguenti aspetti non sono obbligatori, ma contribuiscono a migliorare la valutazione complessiva:

- uso di più collezioni in modo coordinato;
- interfacce (interface) o classi astratte;
- uso di enum;
- suddivisione del codice in package;
- validazione più accurata degli input;
- gestione di report o statistiche;
- codice ben commentato nelle parti davvero importanti;
- progetto particolarmente ordinato e ben progettato.

4. Materiale da consegnare nell'attività Teams

1. Intera cartella del progetto (file .zip)

La cartella deve contenere tutto il materiale necessario al funzionamento:

- file .java;
- istruzioni essenziali per compilazione ed esecuzione;
- Diagramma UML delle classi.

2. Foglio delle ore di lavoro

Deve essere indicato in modo sintetico il tempo dedicato al progetto.
Nel caso di lavoro a coppie, il foglio deve riportare il contributo di entrambi i componenti.

3. Relazione o presentazione

Il documento deve contenere almeno:

- titolo del progetto;
- nome dello studente o dei due componenti;
- descrizione sintetica dell'idea sviluppata;
- struttura generale del programma;
- classi principali utilizzate;
- collezioni utilizzate e motivazione della scelta;
- operazioni principali realizzate;
- difficoltà incontrate e soluzioni adottate.

5. Esposizione Orale

Dopo la consegna può essere richiesto un colloquio orale individuale, anche nel caso di progetto svolto in coppia.

Durante l'orale lo studente deve saper:

- spiegare l'idea generale del progetto;
- descrivere la struttura delle classi;
- giustificare l'uso dell'ereditarietà;
- spiegare le collezioni utilizzate;
- mostrare il funzionamento del menu;
- descrivere il salvataggio o caricamento da file;
- spiegare come sono stati gestiti gli errori;
- illustrare le parti di codice più importanti.

6. Griglia di valutazione

Competenza di riferimento: saper realizzare un'applicazione Java strutturata, funzionante e coerente, usando correttamente la programmazione orientata agli oggetti.

Voto	Descrittore
0-4	progetto molto incompleto, non funzionante oppure quasi privo dei requisiti richiesti
5	progetto essenziale ma con lacune evidenti, struttura debole o diversi requisiti mancanti
6	progetto sufficiente, funzionante nelle parti principali, con uso base di classi e collezioni
7	progetto completo nei requisiti obbligatori, ordinato e ben svolto
8	progetto ben strutturato, con buona autonomia, uso corretto di OOP, collezioni, file ed eccezioni
9-10	voto possibile solo con orale positivo, in presenza di piena comprensione del codice, buone scelte progettuali e padronanza reale del lavoro svolto

7. Suggerimenti operativi

- scegliere un'idea semplice ma realizzabile bene;
- definire chiaramente attributi, metodi e relazioni tra classi;
- costruire prima il modello dati, poi il menu;
- sviluppare e testare il programma a piccoli passi;
- provare ogni funzione prima di passare alla successiva;
- curare i nomi di classi, metodi e variabili;
- evitare codice duplicato;